

Documentation:Airline Management System (AMS)

Project Overview

Project Title:

- Airline Management System

Purpose:

- To design and implement an Angular-based Airline Management System
- Enables users to search, book, and cancel tickets
- Enforces role-based authentication
- Provides separate admin and user modules
- Allows secure management of flights, schedules, and bookings

Target Users:

- User
 - Admin
-

Goal:

- User registration and authentication
- Flight search, booking, and cancellation
- Role-based access (User/Admin)
- Flight and booking management by Admin

System Architecture

Tech Stack:

- **Language/Framework:** Angular 18
 - **Database:** JSON file (`data.json`) for storing and retrieving mock data
 - **Authentication:** Role-based access control using JWT
 - **API Communication:** REST APIs
-

Model Module (Data & CRUD Logic)

- Defines **data structures** for Flights, Users, and Bookings.
- Handles **CRUD operations**:
 - **Flights** → create, update, delete flight details.
 - **Users** → register, authenticate, manage profile.
 - **Bookings** → store, update, cancel user bookings.

View Module

User View

- **Home Page** → Landing page with options to search flights and navigate to login/signup.

- **Sign In / Sign Up** → Secure authentication for users.
 - **Welcome Page** → Greeting page after successful login and interface to search flights.
 - **Profile Page** → Displays user details and booking history.
 - **Booking Page** → Interface , book, and cancel flights.
-

Admin View

Central control panel for managing the system.

- **Flight Management Tab** → Add, update, and delete flight details.
 - **User Management Tab** → View registered users.
 - **Booking Management Tab** → View and manage all bookings made by users.
-

Sample Endpoints

Endpoint	Method	Role	Description
/api/signin	POST	User / Admin	Authenticate user and return JWT token
/api/signup	POST	User	Register a new user account
/api/flights	GET	User / Admin	Retrieve list of available flights

<code>/api/flights</code>	POST	Admin	Add a new flight
<code>/api/flights/{flightId}</code>	PUT	Admin	Update flight details
<code>/api/flights/{flightId}</code>	DELETE	Admin	Delete a flight
<code>/api/bookings</code>	POST	Normal User	Book a ticket for a selected flight
<code>/api/bookings/{bookingId}</code>	DELETE	Normal User	Cancel a booking
<code>/api/bookings/user/{userId}</code>	GET	Normal User	View all bookings of a specific user
<code>/api/bookings</code>	GET	Admin	View all user bookings

Acceptance Criteria

- Users can register and log in via REST API with JWT authentication
 - Normal Users can search flights, book tickets, and cancel bookings
 - Admin can add, update, and delete flights
 - Admin can view all bookings; Users can view their own bookings
 - Role-based access is enforced (User vs Admin)
 - Data persistence through `data.json` mock database
 - API responses return proper status codes and messages
-

Repository Structure

Airlines_project/

```
|— public/          # Angular public assets
|— src/
|  |— app/          # Core Angular app
|  |— models/       # booking, flight, user models & services
|  |— services/     # guards, auth, API, token interceptor
|  |— views/        # UI components/views
|  └— app.module.ts # Root module
|— authMiddleware.js # JWT middleware
|— data.json / data.js # Mock database
|— server.js        # Express backend
|— angular.json, tsconfig*.json, package*.json
|— index.html, main.ts, styles.css
└— README.md, .gitignore, .editorconfig
```